

Double-click (or enter) to edit

1

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import chi2_contingency
import seaborn as sns
from random import randrange, uniform
```

2

```
Train_Data = pd.read_csv('/content/train_cab_fare.csv', nrows=500000, header=0)
if 'key' in Train_Data.columns:
    Train_Data = Train_Data.drop(columns=['key'])
```

3

```
Train_Data.head()
```



	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count
0	4.5	2009-06-15 17:26:21 UTC	-73.844311	40.721319	-73.841610	40.712278	1.0
1	16.9	2010-01-05 16:52:16 UTC	-74.016048	40.711303	-73.979268	40.782004	1.0
2	5.7	2011-08-18 00:35:00 UTC	-73.982738	40.761270	-73.991242	40.750562	2.0
3	7.7	2012-04-21	-73.887400	40.730440	-73.884567	40.750000	1.0

4

```
Train_Data['fare_amount'] = Train_Data['fare_amount'].apply(pd.to_numeric,
errors='coerce')
Train_Data['fare_amount'] = Train_Data['fare_amount'].replace({0:np.nan})
Train_Data['passenger_count'] = Train_Data['passenger_count'].fillna(0)
Train_Data['passenger_count'] = Train_Data['passenger_count'].astype(int)
Train_Data['passenger_count'] = Train_Data['passenger_count'].replace({0:
np.nan})
Train_Data['pickup_longitude'] = Train_Data['pickup_longitude'].replace({0:np.nan})
Train_Data['pickup_latitude'] = Train_Data['pickup_latitude'].replace({0:np.nan})
Train_Data['dropoff_longitude'] = Train_Data['dropoff_longitude'].replace({0:np.nan})
Train_Data['dropoff_latitude'] = Train_Data['dropoff_latitude'].replace({0:np.nan})
```

5

```
Train_Data['fare_amount'] = Train_Data['fare_amount'].apply(pd.to_numeric,
errors='coerce')
Train_Data['fare_amount'] = Train_Data['fare_amount'].replace({0:np.nan})
Train_Data['passenger_count'] = Train_Data['passenger_count'].fillna(0)
Train_Data['passenger_count'] = Train_Data['passenger_count'].astype(int)
Train_Data['passenger_count'] = Train_Data['passenger_count'].replace({0:
np.nan})
Train_Data['pickup_longitude'] = Train_Data['pickup_longitude'].replace({0:np.nan})
Train_Data['pickup_latitude'] = Train_Data['pickup_latitude'].replace({0:np.nan})
Train_Data['dropoff_longitude'] = Train_Data['dropoff_longitude'].replace({0:np.nan})
Train_Data['dropoff_latitude'] = Train_Data['dropoff_latitude'].replace({0:np.nan})
```

6

```
Train_Data.head()
```

	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count
0	4.5	2009-06-15 17:26:21 UTC	-73.844311	40.721319	-73.841610	40.712278	1.0
1	16.9	2010-01-05 16:52:16 UTC	-74.016048	40.711303	-73.979268	40.782004	1.0
2	5.7	2011-08-18 00:35:00 UTC	-73.982738	40.761270	-73.991242	40.750562	2.0
3	7.7	2012-04-21	-73.887100	40.730140	-73.884567	40.750000	1.0

7

Train_Data.shape

(16067, 7)

8

sns.set(style='darkgrid',palette='Set1')

9

```
plt.figure(figsize=(20,20))
plt.subplot(321)
_ = sns.distplot(Train_Data['fare_amount'],bins=50)
plt.subplot(322)
_ = sns.distplot(Train_Data['pickup_longitude'],bins=50)
plt.subplot(323)
_ = sns.distplot(Train_Data['pickup_latitude'],bins=50)
plt.subplot(324)
_ = sns.distplot(Train_Data['dropoff_longitude'],bins=50)
plt.subplot(325)
_ = sns.distplot(Train_Data['dropoff_latitude'],bins=50)
plt.show()
```



<ipython-input-10-29d966f78318>:3: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
_ = sns.distplot(Train_Data['fare_amount'],bins=50)
```

<ipython-input-10-29d966f78318>:5: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
_ = sns.distplot(Train_Data['pickup_longitude'],bins=50)
```

<ipython-input-10-29d966f78318>:7: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
_ = sns.distplot(Train_Data['pickup_latitude'],bins=50)
```

<ipython-input-10-29d966f78318>:9: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
_ = sns.distplot(Train_Data['dropoff_longitude'],bins=50)
```

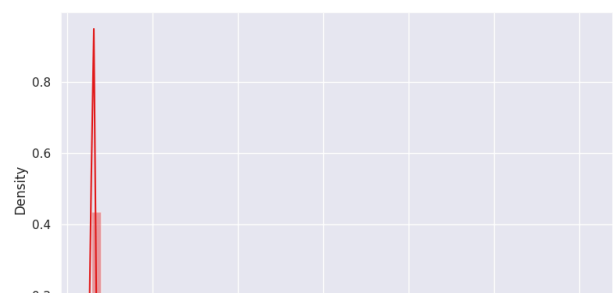
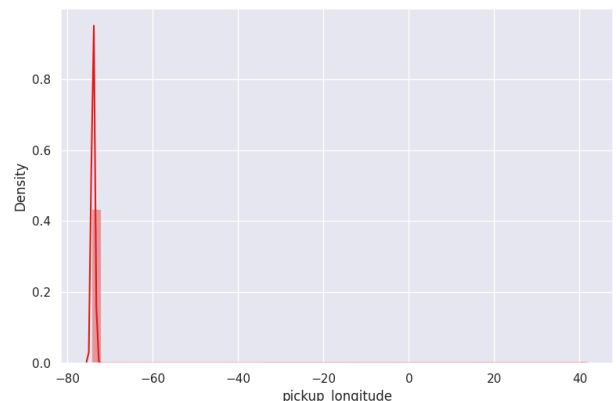
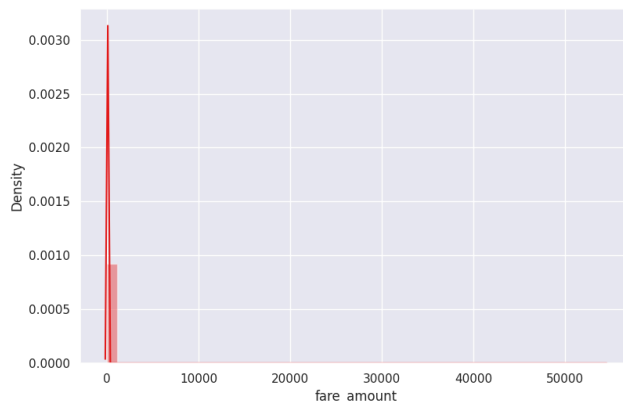
<ipython-input-10-29d966f78318>:11: UserWarning:

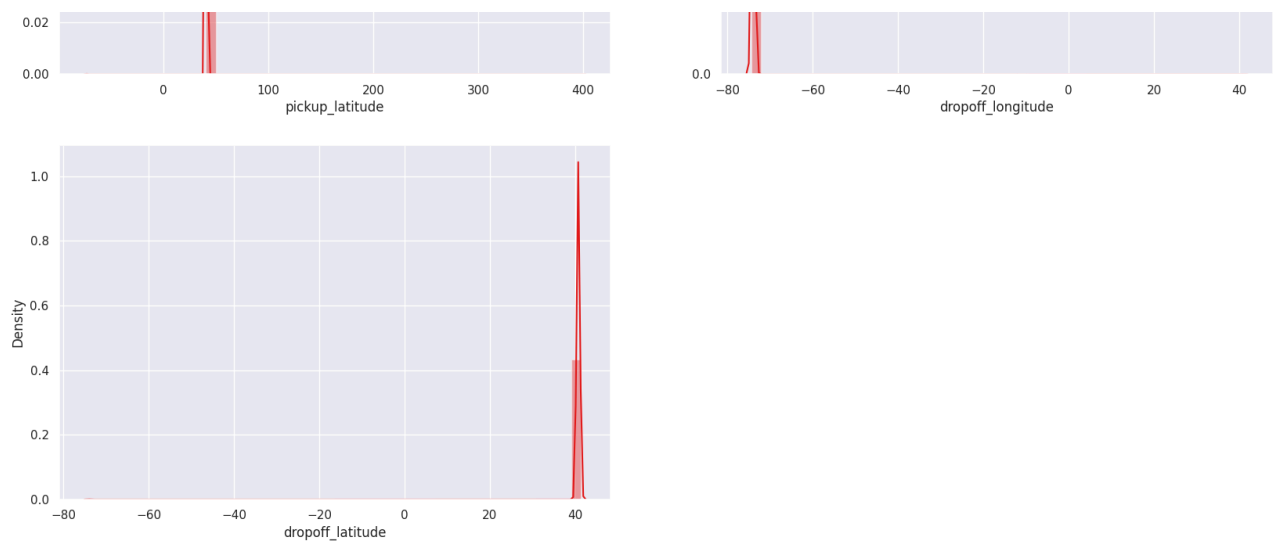
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
_ = sns.distplot(Train_Data['dropoff_latitude'],bins=50)
```





10. calculate missing values

```
missing_val = pd.DataFrame(Train_Data.isnull().sum())

missing_val = missing_val.reset_index()

missing_val = missing_val.rename(columns = {'index': 'Variables', 0: 'count'})

missing_val['Missing_percentage'] = (missing_val['count']/len(Train_Data)*100)

missing_val = missing_val.sort_values('Missing_percentage', ascending =
False).reset_index(drop = True)
print(missing_val)
```

```

Variables  count  Missing_percentage
0  pickup_longitude  315      1.960540
1  pickup_latitude  315      1.960540
2  dropoff_longitude  314      1.954316
3  dropoff_latitude  312      1.941868
4  passenger_count  113      0.703305
5    fare_amount    26      0.161822
6  pickup_datetime     0      0.000000
```

11

```
Train_Data.describe()
```

```

fare_amount  pickup_longitude  pickup_latitude  dropoff_longitude  dropoff_latitude  passenger_count
count  16041.000000      15752.000000      15752.000000      15753.000000      15954.000000      15954.000000
mean    15.015940      -73.911859      40.712919      -73.906698      40.688014      2.634574
std    430.474347      2.651364      3.877557      2.702914      2.624540      60.954312
min     -3.000000     -74.438233     -74.006893     -74.429332     -74.006377      1.000000
25%      6.000000     -73.992386      40.736548     -73.991377      40.736258      1.000000
50%      8.500000     -73.982040      40.753299     -73.980567      40.754223      1.000000
75%     12.500000     -73.968079      40.767800     -73.965360      40.768311      2.000000
max    54343.000000      40.766125      401.083332      40.802437      41.366138      5345.000000
```

12

```
Train_Data['fare_amount'] =Train_Data['fare_amount'].fillna(Train_Data['fare_amount'].mean())
Train_Data['pickup_longitude']=Train_Data['pickup_longitude'].fillna(Train_Data['pickup_longitude'].mean())
Train_Data['pickup_latitude']=Train_Data['pickup_latitude'].fillna(Train_Data['pickup_latitude'].mean())
Train_Data['dropoff_longitude']=Train_Data['dropoff_longitude'].fillna(Train_Data['dropoff_longitude'].mean())
Train_Data['dropoff_latitude']=Train_Data['dropoff_latitude'].fillna(Train_Data['dropoff_latitude'].mean())
```

```
Train_Data['passenger_count'] =Train_Data['passenger_count'].fillna(int(Train_Data['passenger_count'].mode()))
```

```
<ipython-input-18-7bc7daa9b18c>:1: FutureWarning: Calling int on a single element Series is deprecated and will raise a Typ
Train_Data['passenger_count'] =Train_Data['passenger_count'].fillna(int(Train_Data['passenger_count'].mode()))
```

13

```
Train_Data.head()
```

```

fare_amount  pickup_datetime  pickup_longitude  pickup_latitude  dropoff_longitude  dropoff_latitude  passenger_count
0          4.5      2009-06-15 17:26:21 UTC      -73.844311      40.721319      -73.841610      40.712278          1.0
1         16.9      2010-01-05 16:52:16 UTC      -74.016048      40.711303      -73.979268      40.782004          1.0
2          5.7      2011-08-18 00:35:00 UTC      -73.982738      40.761270      -73.991242      40.750562          2.0
3          7.7      2012-04-21      73.887488      40.738148      73.881567      40.758888          1.0
```

14

```
Train_Data.isnull()
```

```

fare_amount  pickup_datetime  pickup_longitude  pickup_latitude  dropoff_longitude  dropoff_latitude  passenger_coun
0          False          False          False          False          False          False          False
1          False          False          False          False          False          False          False
2          False          False          False          False          False          False          False
3          False          False          False          False          False          False          False
4          False          False          False          False          False          False          False
...          ...          ...          ...          ...          ...          ...          ...
16062        False          False          False          False          False          False          False
16063        False          False          False          False          False          False          False
16064        False          False          False          False          False          False          False
16065        False          False          False          False          False          False          False
16066        False          False          False          False          False          False          False
```

15

```
Train_Data=Train_Data.dropna()
```

16

```
convert_dic={'fare_amount' : 'float','passenger_count': 'int'}
Train_Data=Train_Data.astype(convert_dic)
```

17

```
Train_Data.shape
```

↩ (16067, 7)

18

```
df = Train_Data.copy()
Train_Data = Train_Data.copy()
```

19

```
Train_Data.loc[Train_Data['fare_amount']<0 , 'fare_amount']=np.nan
Train_Data.loc[Train_Data['fare_amount'] > 30, 'fare_amount']=np.nan
Train_Data=Train_Data.dropna()
```

20

```
Train_Data.loc[Train_Data['passenger_count'] > 8,'passenger_count'] = np.nan
```

Start coding or [generate](#) with AI.

21.save numeric data names

```
import numpy as np
coutliers = ['pickup_longitude', 'pickup_latitude', 'dropoff_longitude', 'dropoff_latitude']
for col in coutliers:
    # Convert the column to numeric, errors='coerce' will replace invalid values with NaN
    Train_Data[col] = pd.to_numeric(Train_Data[col], errors='coerce')

    # Calculate percentiles on the numeric column
    q75, q25 = np.percentile(Train_Data[col], [75, 25])

    # you can print or use the q75, q25 values as needed for further calculations
    print(f"Column: {col}, Q75: {q75}, Q25: {q25}")
```

↩ Column: pickup_longitude, Q75: -73.968165, Q25: -73.992417
Column: pickup_latitude, Q75: 40.766992, Q25: 40.735706
Column: dropoff_longitude, Q75: -73.965395, Q25: -73.99134
Column: dropoff_latitude, Q75: 40.767682, Q25: 40.735418
<ipython-input-27-c52d468b5eaf>:5: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-or-copy
Train_Data[col] = pd.to_numeric(Train_Data[col], errors='coerce')

22

```
Train_Data['pickup_latitude'] =Train_Data['pickup_latitude'].fillna(Train_Data['pickup_latitude'].mean())
```

↩ <ipython-input-28-a54d1a5e5b55>:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-or-copy
Train_Data['pickup_latitude'] =Train_Data['pickup_latitude'].fillna(Train_Data['pickup_latitude'].mean())

```
Train_Data['pickup_longitude'] =Train_Data['pickup_longitude'].fillna(Train_Data['pickup_longitude'].mean())
Train_Data['pickup_latitude'] =Train_Data['pickup_latitude'].fillna(Train_Data['pickup_latitude'].mean())
Train_Data['dropoff_longitude'] =Train_Data['dropoff_longitude'].fillna(Train_Data['dropoff_longitude'].mean())
Train_Data['dropoff_latitude'] =Train_Data['dropoff_latitude'].fillna(Train_Data['dropoff_latitude'].mean())
#imputed with mode for categorical variables
Train_Data['passenger_count'] =Train_Data['passenger_count'].fillna(int(Train_Data['passenger_count'].mode()))
```

↩ <ipython-input-29-9518c5e5afc5>:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
 Train_Data['pickup_longitude'] =Train_Data['pickup_longitude'].fillna(Train_Data['pickup_longitude'].mean())
 <ipython-input-29-9518c5e5afc5>:2: SettingWithCopyWarning:
 A value is trying to be set on a copy of a slice from a DataFrame.
 Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
 Train_Data['pickup_latitude'] =Train_Data['pickup_latitude'].fillna(Train_Data['pickup_latitude'].mean())
 <ipython-input-29-9518c5e5afc5>:3: SettingWithCopyWarning:
 A value is trying to be set on a copy of a slice from a DataFrame.
 Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
 Train_Data['dropoff_longitude'] =Train_Data['dropoff_longitude'].fillna(Train_Data['dropoff_longitude'].mean())
 <ipython-input-29-9518c5e5afc5>:4: SettingWithCopyWarning:
 A value is trying to be set on a copy of a slice from a DataFrame.
 Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
 Train_Data['dropoff_latitude'] =Train_Data['dropoff_latitude'].fillna(Train_Data['dropoff_latitude'].mean())
 <ipython-input-29-9518c5e5afc5>:6: FutureWarning: Calling int on a single element Series is deprecated and will raise a Ty
 Train_Data['passenger_count'] =Train_Data['passenger_count'].fillna(int(Train_Data['passenger_count'].mode()))
 <ipython-input-29-9518c5e5afc5>:6: SettingWithCopyWarning:
 A value is trying to be set on a copy of a slice from a DataFrame.
 Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
 Train_Data['passenger_count'] =Train_Data['passenger_count'].fillna(int(Train_Data['passenger_count'].mode()))

23

```
Train_Data['passenger_count']=Train_Data['passenger_count'].astype('int')
Train_Data['passenger_count']=Train_Data['passenger_count'].astype('category')
```

 <ipython-input-30-e1aca1e36f4f>:1: SettingWithCopyWarning:
 A value is trying to be set on a copy of a slice from a DataFrame.
 Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
 Train_Data['passenger_count']=Train_Data['passenger_count'].astype('int')
 <ipython-input-30-e1aca1e36f4f>:2: SettingWithCopyWarning:
 A value is trying to be set on a copy of a slice from a DataFrame.
 Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
 Train_Data['passenger_count']=Train_Data['passenger_count'].astype('category')

24

```
def haversine(lat1, lon1, lat2, lon2, to_radians=True, earth_radius=6371):
    if to_radians:
        lat1, lon1, lat2, lon2 = np.radians([lat1, lon1, lat2, lon2])
    a = np.sin((lat2-lat1)/2.0)*2 + np.cos(lat1) * np.cos(lat2) * np.sin((lon2-lon1)/2.0)*2
    return earth_radius * 2 * np.arcsin(np.sqrt(a))
```

25

```
Train_Data['dist'] = haversine( Train_Data['pickup_latitude'],
Train_Data['pickup_longitude'],
Train_Data['dropoff_latitude'], Train_Data['dropoff_longitude'])
```

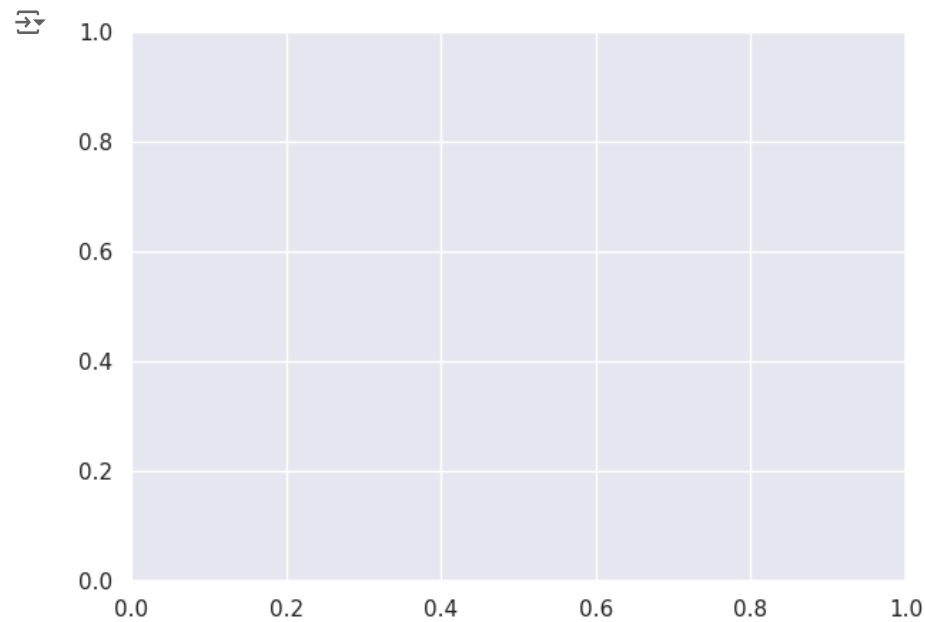
 <ipython-input-31-f029efc45ffb>:5: RuntimeWarning: invalid value encountered in sqrt
 return earth_radius * 2 * np.arcsin(np.sqrt(a))
 <ipython-input-32-b7271d59632f>:1: SettingWithCopyWarning:
 A value is trying to be set on a copy of a slice from a DataFrame.
 Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
 Train_Data['dist'] = haversine(Train_Data['pickup_latitude'],

```
numeric=['fare_amount','pickup_longitude','pickup_latitude','dropoff_longitude','dropoff_latitude', 'dist']
Train_Data_corr = Train_Data.loc[:,numeric]
```

27

```
f, ax = plt.subplots(figsize=(7, 5))
```



```
corr = Train_Data_corr.corr()
print(corr)
```

```
fare_amount    pickup_longitude  pickup_latitude \
fare_amount      1.000000      0.013823    -0.014694
pickup_longitude  0.013823      1.000000    -0.643841
pickup_latitude  -0.014694    -0.643841      1.000000
dropoff_longitude  0.014470      0.978328    -0.629991
dropoff_latitude  -0.010117    -0.985795      0.642260
dist              0.470051     -0.000564      0.031440

fare_amount    dropoff_longitude  dropoff_latitude    dist
fare_amount      0.014470      -0.010117    0.470051
pickup_longitude  0.978328      -0.985795   -0.000564
pickup_latitude  -0.629991      0.642260    0.031440
dropoff_longitude  1.000000      -0.964482    0.160754
dropoff_latitude  -0.964482      1.000000    0.000926
dist              0.160754      0.000926    1.000000
```

```
sns.heatmap(corr, mask=np.zeros_like(corr, dtype=bool),cmap=sns.diverging_palette(220, 10, as_cmap=True),square=True, ax=ax)
```

```
<Axes: >
```

28

```
Train_Data=Train_Data[np.logical_and(Train_Data['pickup_longitude'] !=
Train_Data['dropoff_longitude'],
Train_Data['pickup_latitude'] !=
Train_Data['dropoff_latitude'])]
```

29

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
```

30


```
train1, test1 = train_test_split(Train_Data, test_size=0.2)
```

31

```
numerical_features = ['passenger_count', 'pickup_longitude', 'pickup_latitude', 'dropoff_longitude', 'dropoff_latitude', 'dist']
RF_model = RandomForestRegressor(n_estimators=100).fit(train1[numerical_features], train1['fare_amount'])
```

32

```
RF_model
```

 RandomForestRegressor ⓘ ?
RandomForestRegressor()

33

```
RF_Predictions = RF_model.predict(test1[numerical_features])
```

34

```
def MAPE(y_true, y_pred):
    mape = np.mean(np.abs((y_true - y_pred) / y_true))*100
    return mape
```

35

```
MAPE(test1.iloc[:,0], RF_Predictions)
```

 21.858239730136475

36. #Error 21.826 #Accuracy 78.174

37

```
values=['fare_amount', 'pickup_longitude', 'pickup_latitude', 'dropoff_longitude', 'dropoff_latitude', 'dist']
```

38

```
linear_Data = Train_Data[values]
```

39

```
cat_names = ['passenger_count']
for i in cat_names:
    temp = pd.get_dummies(Train_Data[i], prefix= i)
    linear_Data = linear_Data.join(temp)
```

40

```
linear_Data.shape
```

 (15079, 12)

41

```
linear_Data.head()
```

	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	dist	passenger_count_1	pass
0	4.5	-73.844311	40.721319	-73.841610	40.712278	NaN		True
1	16.9	-74.016048	40.711303	-73.979268	40.782004	510.200003		True
2	5.7	-73.982738	40.761270	-73.991242	40.750562	NaN		False
3	7.7	-73.987130	40.733143	-73.991567	40.758092	251.970908		True
4	5.3	-73.968095	40.768008	-73.956655	40.783762	251.476075		True

42

```
train1, test1 = train_test_split(linear_Data, test_size=0.2)
```

43

```
import statsmodels.api as sm
import pandas as pd
import numpy as np

# Replace infinite values with NaN
train1.replace([np.inf, -np.inf], np.nan, inplace=True)

# Drop rows with any missing values
train1.dropna(inplace=True)

# Re-fit the model after handling missing values
model = sm.OLS(train1.iloc[:, 0].astype(float), train1.iloc[:, 1:12].astype(float)).fit()
```

44

```
model.summary()
```



OLS Regression Results						
Dep. Variable:	fare_amount	R-squared:	0.511			
Model:	OLS	Adj. R-squared:	0.510			
Method:	Least Squares	F-statistic:	634.9			
Date:	Wed, 13 Nov 2024	Prob (F-statistic):	0.00			
Time:	13:24:09	Log-Likelihood:	-16482.			
No. Observations:	6080	AIC:	3.299e+04			
Df Residuals:	6069	BIC:	3.306e+04			
Df Model:	10					
Covariance Type: nonrobust						
	coef	std err	t	P> t	[0.025	0.975]
pickup_longitude	7.1604	1.190	6.019	0.000	4.828	9.493
pickup_latitude	54.8949	2.375	23.109	0.000	50.238	59.552
dropoff_longitude	-6.8356	0.116	-59.175	0.000	-7.062	-6.609
dropoff_latitude	-54.5832	2.505	-21.789	0.000	-59.494	-49.672
dist	0.0395	0.001	68.368	0.000	0.038	0.041
passenger_count_1	12.2191	39.502	0.309	0.757	-65.218	89.656
passenger_count_2	12.1282	39.503	0.307	0.759	-65.312	89.568
passenger_count_3	12.2769	39.509	0.311	0.756	-65.175	89.729
passenger_count_4	12.0856	39.508	0.306	0.760	-65.364	89.535
passenger_count_5	12.0760	39.502	0.306	0.760	-65.363	89.515
passenger_count_6	13.5166	39.500	0.342	0.732	-63.917	90.950
Omnibus:	1773.523	Durbin-Watson:	1.964			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	6379.195			
Skew:	1.434	Prob(JB):	0.00			
Kurtosis:	7.118	Cond. No.	6.81e+05			

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 6.81e+05. This might indicate that there are strong multicollinearity or other numerical problems.

45

```
predictions_LR = model.predict(test1.iloc[:,1:12])
```

46

```
MAPE(test1.iloc[:,0], predictions_LR)
```

```
↗ 425.6806460115624
```

```
47. #Error 28.174474283418512
```

✓ Accuracy 71.8255

48

```
pred=(pd.read_csv('/content/test_cab_fare (1).csv', header = 0)).drop(columns="pickup_datetime")
```

```
pred.head()
```

```
↗
```

	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count
0	-73.973320	40.763805	-73.981430	40.743835	1
1	-73.986862	40.719383	-73.998886	40.739201	1
2	-73.982524	40.751260	-73.979654	40.746139	1
3	-73.981160	40.767807	-73.990448	40.751635	1
4	-73.966046	40.789775	-73.988565	40.744427	1

49

```
pred['dist'] = haversine( pred['pickup_latitude'], pred['pickup_longitude'],
pred['dropoff_latitude'], pred['dropoff_longitude'])
pred['fare_amount']=0
pred['passenger_count']=pred['passenger_count'].astype('category')
```

```
↗ <ipython-input-31-f029efc45ffb>:5: RuntimeWarning: invalid value encountered in sqrt
return earth_radius * 2 * np.arcsin(np.sqrt(a))
```

50

```
RF_model = RandomForestRegressor(n_estimators=10).fit(Train_Data[['passenger_count', 'pickup_longitude', 'pickup_latitude', 'dr
```

```
pred['fare_amount'] = RF_model.predict(pred[['passenger_count', 'pickup_longitude', 'pickup_latitude', 'dropoff_longitude', 'dr
```

51

```
pred.head()
```

```
↗
```

	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count	dist	fare_amount
0	-73.973320	40.763805	-73.981430	40.743835	1	NaN	8.31
1	-73.986862	40.719383	-73.998886	40.739201	1	191.292144	7.55
2	-73.982524	40.751260	-73.979654	40.746139	1	NaN	4.91
3	-73.981160	40.767807	-73.990448	40.751635	1	NaN	8.81
4	-73.966046	40.789775	-73.988565	40.744427	1	NaN	13.24

52